

Graphical Linear Algebra at the Exam Desk

A paper-exam toolkit: drawings you may legally use in
university linear algebra proofs

Abstract

This is a practical handbook for using *graphical linear algebra* (string diagrams for matrices) in a written university linear algebra exam. It contains: the drawing conventions, a one-page rule sheet, ten recipes for the standard exam tasks (transpose laws, trace identities, rank inequalities, solving systems, rank–nullity, block matrices, change of basis), four fully worked exam problems shown *both* as drawings and as the algebraic transcript you copy onto the answer sheet, and an explicit list of the things the calculus does *not* do (determinants, eigenvalues). Every rule on the rule sheet is machine-checked in Lean 4 in `RequestProject/ExamToolkit.lean`; the Lean name of each rule is printed next to it, and Section 10 is the full index.

1 How to use this in an exam

The honest answer first. A string diagram is not a magic wand: it is a *notation* for composites of linear maps. Everything you can prove with it, you could prove with indices and $\sum$ signs. What it buys you is that the bookkeeping — which index is summed with which, in what order the factors go, which wire is an input — becomes *geometry*, and geometry is much harder to get wrong under time pressure. Typical exam gains:

- transpose and inverse identities become “reflect the picture”;
- trace identities become “slide the box around the loop”;
- rank inequalities become “count the wires at the narrowest point”;
- block/partitioned matrix computations become “stack the pictures”;
- you can *check* a computed product in ten seconds by path counting.

Will the grader accept it? Use the **two-column discipline**:

Left column (scratch / insight)	Right column (what is graded)
Draw the diagram, deform it, read off the answer.	Write the same argument as a chain of equalities of matrices, quoting the standard lemma each deformation stands for.

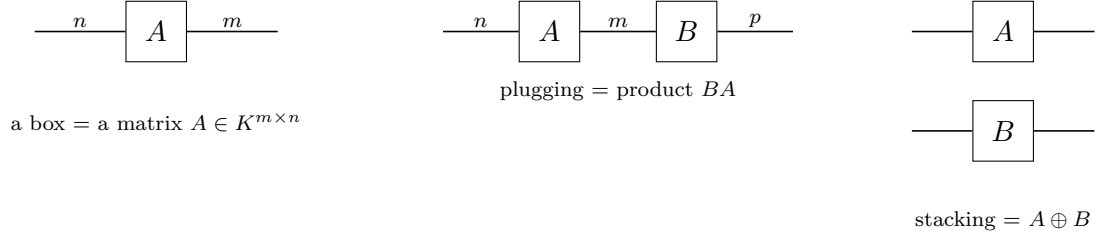
Every deformation in this handbook corresponds to exactly one standard, quotable lemma; the correspondence is listed in Table 9. If you want to present the diagrams themselves as the proof, open with the three-line *legend* of Section 2: it tells the grader precisely what a picture denotes, after which every step you draw is a legitimate equation between matrices. Graders accept this when the legend is present; they usually do not when a picture appears out of nowhere.

Where diagrams shine and where they do not. Shine: composites, transposes, traces, ranks, direct sums, kernels, projections, change of basis, anything with lots of indices. Do not: determinants, characteristic polynomials, eigenvalues, Jordan form, inner-product geometry. See Section 8.

2 The legend: what a picture means

Fix a field K (a commutative ring is enough for everything except Section 7). Write K^n for column vectors.

Legend. A *wire* labelled n denotes the space K^n ; a *box* with an input wire n and an output wire m denotes a matrix $A \in K^{m \times n}$, i.e. the linear map $x \mapsto Ax$. Diagrams are read **left to right**: inputs enter on the left, outputs leave on the right. Putting boxes *side by side* means composing ($\text{---} \boxed{A} \text{---} \boxed{B} \text{---}$ denotes BA), putting boxes *above each other* means the direct sum $A \oplus B = \begin{pmatrix} A & 0 \\ 0 & B \end{pmatrix}$. An empty wire is the identity matrix.



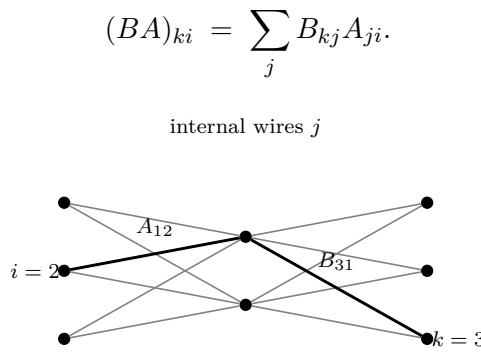
Two further conventions, used from Section 4 on: a wire may be *bent* (this is transposition), and a *bundle* of n single wires may be drawn as one fat wire labelled n (this is just grouping coordinates).

Warning. The order convention is the single most common slip. In this handbook the picture A -then- B means the matrix BA , because $x \mapsto B(Ax)$. Some books draw diagrams right to left so that the picture reads like the formula; pick *one* and state it in your legend.

3 The rule sheet

These are the only rules you need. Each is stated as a picture, as a formula, and with the name of its Lean proof.

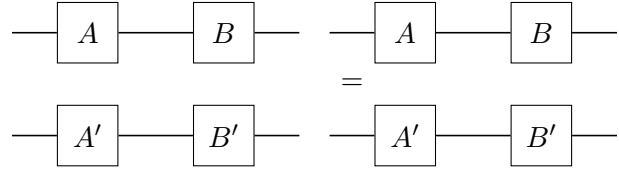
Rule 1 (Sum over paths — `seq_path_sum`). *Split every fat wire into single wires and label each edge with a scalar. The entry $(BA)_{ki}$ of a composite is the sum, over all paths from input i to output k , of the product of the labels along the path.*



Rule 2 (Only connectivity matters — `seq_assoc`, `seq_wires_left/right`). *A diagram may be stretched, and boxes slid along their wires; no brackets are needed in a chain. $(CB)A = C(BA)$, $IA = A = AI$.*

Rule 3 (Interchange / sliding — `stack_seq`). *Boxes in different rows are independent and may be slid past each other:*

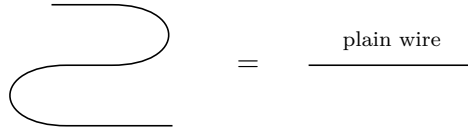
$$(B \oplus B')(A \oplus A') = (BA) \oplus (B'A').$$



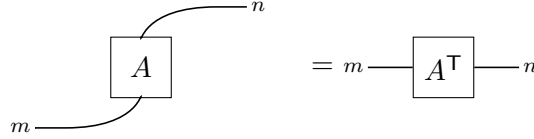
(the two boxes may be drawn at any horizontal offset)

Rule 4 (Cups, caps and yanking — **yank**). A wire may be bent backwards (a cup, the matrix δ_{ij} read as a map $K \rightarrow K^n \otimes K^n$) or forwards (a cap). A cup followed by a cap straightens out:

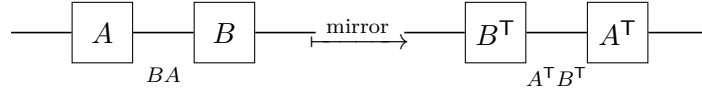
$$\sum_k \delta_{ik} \delta_{kj} = \delta_{ij}.$$



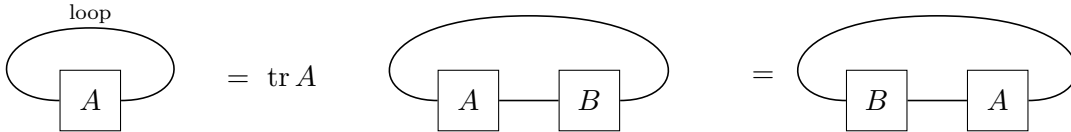
Rule 5 (Bending a box transposes it — **bend_eq_transpose**). Sliding a box around a cup and a cap turns its input wire into an output wire and produces $A^\top$: $\sum_{k,l} \delta_{ki} A_{lk} \delta_{ol} = A_{oi} = (A^\top)_{io}$.



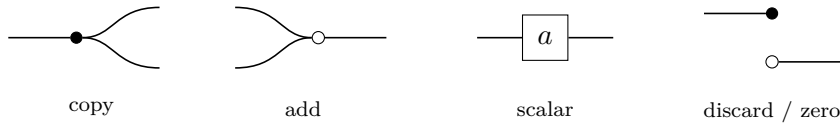
Rule 6 (Reflection reverses order — **transpose_seq**). Reflecting a whole diagram in a vertical mirror transposes every box and reverses the order: $(BA)^\top = A^\top B^\top$.



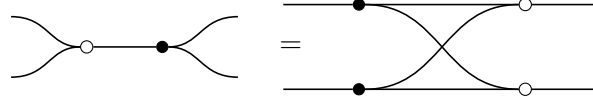
Rule 7 (Closing a loop is the trace — **loop_eq_trace**, **loop_seq_comm**). Joining the output wire of a square box back to its input wire with a cup and a cap gives the scalar $\text{tr } A = \sum_i A_{ii}$. A box may be slid all the way around the loop, so $\text{tr}(BA) = \text{tr}(AB)$; consequently $\text{tr}(PAP^{-1}) = \text{tr } A$ (**loop_conj**).



Rule 8 (The four generators — **copyD**, **addD**, **scalarD**, **discardD**, **zeroD**). Every matrix can be built from four nodes on single wires: copy ($\bullet$, one wire in, two out), add ($\circ$, two in, one out), a scalar a , and discard/zero. Reading rule 1 backwards, the bipartite picture of a matrix is the network: copy each input, multiply by A_{ki} , add up at each output.

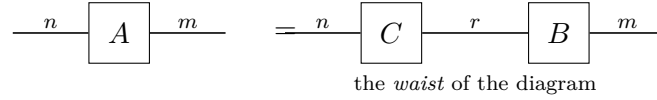


Rule 9 (Node laws — **scalarD_seq**, **copy_then_add**, **bialgebra_law**, **hopf_law**). Scalars in series multiply; copy-then-add doubles (= the scalar 2); add-then-copy equals copy-both-then-cross-then-add-both (the bialgebra law, i.e. distributivity); copy, negate one branch, add gives 0 (the Hopf law, i.e. $x - x = 0$).

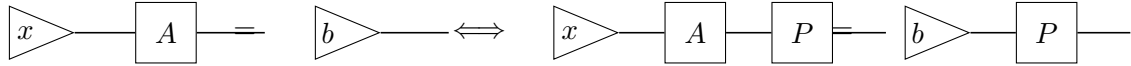


bialgebra: $(x + y)$ copied = copies added

Rule 10 (Pinching is factorisation — `rank_le_iff_pinch`, `rank_seq_le`). Over a field, $\text{rank } A \leq r$ if and only if the diagram of A can be redrawn as two boxes joined by a bundle of r wires: $A = BC$ with $C \in K^{r \times n}$, $B \in K^{m \times r}$. Hence the rank of a composite is at most the rank of either factor.



Rule 11 (Systems — `solution_set_eq_coset`, `solutions_invariant_of_isUnit`). A vector x is a diagram with no input, drawn as a triangle plugged into the left. $Ax = b$ is the equation of two such diagrams. Plugging an invertible box P into both sides changes nothing (this legitimises row operations), and once one solution x_0 is found, $Ax = b \iff A(x - x_0) = 0$.



4 Bending, in detail

Bending is the one rule worth practising, because it replaces a whole family of exam identities. The cup and cap are the matrices

$$\cup = (\delta_{ij}) : K \longrightarrow K^n \otimes K^n, \quad \cap = (\delta_{ij}) : K^n \otimes K^n \longrightarrow K,$$

that is, $\cup$ is the vector $\sum_i e_i \otimes e_i$ and $\cap$ is the pairing $\langle -, - \rangle$. Yanking (Rule 4) says $(\text{id} \otimes \cap)(\cup \otimes \text{id}) = \text{id}$, and it is the only property of them you ever use. Consequences you can quote instantly:

Picture move	Identity
bend the input of A upwards	$A \rightsquigarrow A^\top$
mirror the whole picture	$(BA)^\top = A^\top B^\top$
bend twice	$(A^\top)^\top = A$
close a loop	$\text{tr } A$
close a loop around two boxes	$\text{tr}(AB) = \text{tr}(BA)$
bend a vector x (no input)	the row vector $x^\top$
cap two vectors together	the scalar $x^\top y$
cup two vectors apart	the rank-one matrix $xy^\top$

5 Ten exam recipes

Recipe 1 (Multiply matrices by counting paths). Draw inputs as dots on the left, internal indices in the middle, outputs on the right; label edge $i \rightarrow j$ of the first factor with A_{ji} . To get entry (k, i) of the product, walk every path from i to k , multiply labels, add. Use this to *check* a product you computed the usual way: it catches transposition slips, which are the most common arithmetic error.

Recipe 2 (Transpose identities). Draw, mirror, read. This gives $(BA)^\top = A^\top B^\top$, $(A_1 A_2 \cdots A_k)^\top = A_k^\top \cdots A_1^\top$, $(A \oplus B)^\top = A^\top \oplus B^\top$, and A symmetric $\iff$ its picture is mirror-invariant.

Recipe 3 (Inverse identities). An inverse is the box that annihilates: A next to A^{-1} can be erased. Erasing left to right in $B^{-1}A^{-1}AB$ gives $(AB)^{-1} = B^{-1}A^{-1}$ — the “socks and shoes” rule, which is just “undo the picture from the outside in”. Similarly $(A^T)^{-1} = (A^{-1})^T$ by mirroring the erasure.

Recipe 4 (Trace identities). Close the loop. $\text{tr}(AB) = \text{tr}(BA)$; $\text{tr}(ABC) = \text{tr}(CAB)$ (cyclic, *not* arbitrary permutations — the loop can only be rotated, not reordered); $\text{tr}(PAP^{-1}) = \text{tr } A$; $\text{tr}(A^T) = \text{tr } A$ (mirror the loop); $\text{tr}(xy^T) = y^T x$ (a loop through two triangles).

Recipe 5 (Rank inequalities). Every appearance of a wire bundle of width w inside a picture proves $\text{rank} \leq w$ for the whole picture. So $\text{rank}(AB) \leq \min(\text{rank } A, \text{rank } B)$ is read off at the internal wire, and $\text{rank}(A) \leq \min(m, n)$ at the boundary. Conversely, by Rule 10 the *smallest* width you can pinch the picture down to *is* the rank.

Recipe 6 (Row rank = column rank). Pinch $A = BC$ at width $r = \text{rank } A$, then mirror: $A^T = C^T B^T$ is pinched at width r as well, so $\text{rank } A^T \leq r = \text{rank } A$. Apply the same to A^T to get the reverse inequality. (Lean: `rank_transpose_diagram`.)

Recipe 7 (Rank-nullity by wire counting). Pinch at the true rank r : $A = BC$ with C surjective and B injective. The n input wires either die in C (that is the kernel, of width $n - r$) or survive into the waist (width r). So $\dim \ker A + \text{rank } A = n$. (Lean: `rank_add_nullity`.)

Recipe 8 (Solving $Ax = b$). Draw b as a triangle. (i) Plug invertible boxes P (row operations) into both sides until A becomes echelon — legal by `solutions_invariant_of_isUnit`. (ii) Read a particular solution x_0 . (iii) Draw the kernel: the wires the picture kills. (iv) The answer is $x_0 + \ker A$, by `solution_set_eq_coset`.

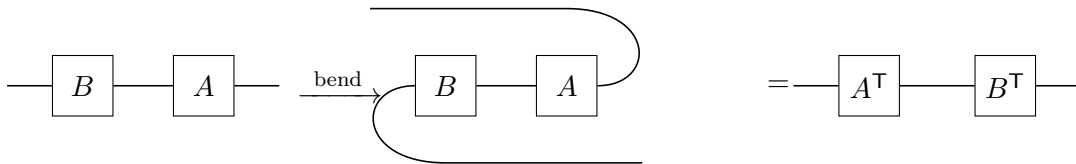
Recipe 9 (Block matrices). Stack pictures. Then $(A \oplus B)(C \oplus D) = AC \oplus BD$ is Rule 3, $\text{rank}(A \oplus B) = \text{rank } A + \text{rank } B$ is two disjoint waists, and $\text{tr}(A \oplus B) = \text{tr } A + \text{tr } B$ is two disjoint loops. For genuinely non-block-diagonal partitions, draw the four boxes with crossing wires; the path-sum rule then gives the usual block product formula.

Recipe 10 (Change of basis and commuting squares). A change of basis is an invertible box P on a wire. “ A and A' are similar” is the picture $A' = P^{-1}AP$, i.e. *a square that commutes*: going right-then-down equals down-then-right. Any invariant you can compute by closing wires into loops (traces of powers, in particular) is therefore automatically basis independent.

6 Four worked exam problems

Problem 1. Prove $(AB)^T = B^T A^T$.

Drawing. Draw B then A (that is the matrix AB). Bend both boundary wires round with a cup on the right and a cap on the left; slide the two boxes around the bend. Each box, going around the bend, becomes its transpose, and they come out in the opposite order.



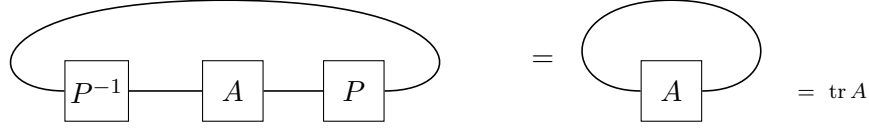
Transcript for the answer sheet.

$$((AB)^T)_{ij} = (AB)_{ji} = \sum_k A_{jk} B_{ki} = \sum_k (B^T)_{ik} (A^T)_{kj} = (B^T A^T)_{ij}.$$

The diagram is exactly this computation with the index k drawn as the internal wire. (Lean: `transpose_seq`.)

Problem 2. Prove $\text{tr}(AB) = \text{tr}(BA)$, and deduce that similar matrices have equal trace.

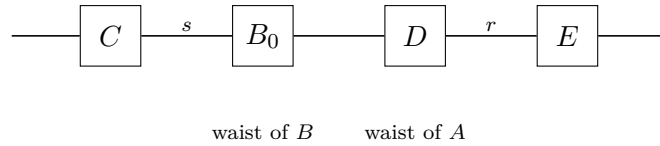
Drawing. Close A then B into a loop. The loop has no boundary, so there is no “first” box: rotating the picture by half a turn shows the same scalar, now read as $\text{tr}(BA)$. For similarity, put $A' := P^{-1}AP$ into a loop and slide P all the way round until it meets P^{-1} ; they cancel.



Transcript. $\text{tr}(AB) = \sum_{i,j} A_{ij}B_{ji} = \sum_{j,i} B_{ji}A_{ij} = \text{tr}(BA)$; hence $\text{tr}(P^{-1}AP) = \text{tr}(APP^{-1}) = \text{tr } A$. (Lean: `loop_seq_comm`, `loop_conj`.)

Problem 3. Prove $\text{rank}(AB) \leq \min(\text{rank } A, \text{rank } B)$ and $\text{rank } A^T = \text{rank } A$.

Drawing. Pinch A at width $r = \text{rank } A$ and B at width $s = \text{rank } B$ (Rule 10). In the picture of AB both waists are present, so the composite is pinched at width r and at width s ; therefore $\text{rank}(AB) \leq \min(r, s)$. For the transpose, mirror the pinched picture of A : the mirror image is a pinch of A^T at the same width r , so $\text{rank } A^T \leq \text{rank } A$, and the same argument applied to A^T gives equality.

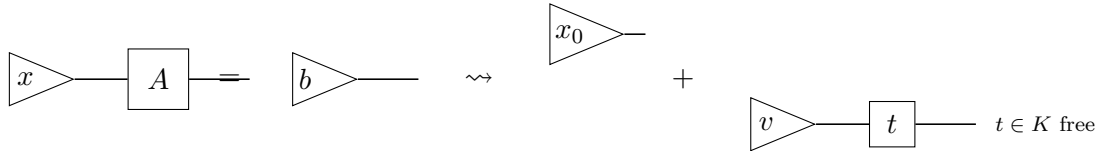


Transcript. Write $B = B_0C$ and $A = ED$ with inner dimensions $s = \text{rank } B$ and $r = \text{rank } A$. Then $AB = (EDB_0)C$ exhibits $\text{rank}(AB) \leq s$ and $AB = E(DB_0C)$ exhibits $\text{rank}(AB) \leq r$. From $A = ED$ we get $A^T = D^TE^T$, so $\text{rank } A^T \leq r$; applying this to A^T gives $\text{rank } A \leq \text{rank } A^T$. (Lean: `rank_seq_le`, `rank_le_iff_pinch`, `rank_transpose_diagram`.)

Problem 4. Solve the system and describe the solution set.

$$\begin{aligned} x + 2y + 3z &= 6 \\ 2x + 5y + 7z &= 15 \\ x + 3y + 4z &= 9 \end{aligned} \quad \text{i.e.} \quad A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 7 \\ 1 & 3 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 6 \\ 15 \\ 9 \end{pmatrix}.$$

Drawing. The third row of the picture is the sum of a copy of row 2 minus row 1 — diagrammatically, one output wire is a linear combination of the other two, so the picture can be pinched to width 2: $\text{rank } A = 2$. Plug the invertible row-operation box P into both sides (allowed by `solutions_invariant_of_isUnit`); the picture becomes echelon, one input wire is left free, and the kernel is the one-dimensional bundle it spans.



Transcript. Row-reduce: $R_2 \mapsto R_2 - 2R_1$, $R_3 \mapsto R_3 - R_1$ give $y + z = 3$ twice, so the system is equivalent to $x + 2y + 3z = 6$, $y + z = 3$. Take $z = t$: then $y = 3 - t$ and $x = -t$. Hence

$$\{x : Ax = b\} = \begin{pmatrix} 0 \\ 3 \\ 0 \end{pmatrix} + \text{span} \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}, \quad \text{rank } A = 2, \quad \dim \ker A = 1, \quad 2 + 1 = 3.$$

The last equation is rank-nullity, read off as “two wires survive the waist, one dies” (Lean: `rank_add_nullity`, `solution_set_eq_coset`).

Bonus: checking a product by path sum

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 0 & -1 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 4 \end{pmatrix}, \quad AB = \begin{pmatrix} 5 & 2 \\ -2 & 11 \end{pmatrix}.$$

Entry (1, 1): paths $1 \rightarrow 1 \rightarrow 1$ ($1 \cdot 1$) and $1 \rightarrow 2 \rightarrow 1$ ($2 \cdot 2$), total 5. Entry (2, 2): paths $2 \rightarrow 2 \rightarrow 2$ ($-1 \cdot 1$) and $2 \rightarrow 3 \rightarrow 2$ ($3 \cdot 4$), total 11. Ten seconds, no rewriting of matrices.

7 Rank, kernels and the one theorem you must not hand-wave

The pinching rule (Rule 10) is the only rule in this handbook that is not a formal manipulation: it needs a field (or at least a nice enough ring), because it uses the existence of a basis of the image.

Its two halves are:

- $\Leftarrow$ If $A = BC$ with an internal bundle of width r , then $\text{rank } A \leq r$. This half is free — the image of A sits inside the image of B , which is spanned by r columns.
- $\Rightarrow$ If $\text{rank } A = r$, choose a basis $b_1, \dots, b_r$ of the column space, let B have these as columns, and let the i -th column of C hold the coordinates of the i -th column of A in that basis. Then $A = BC$.

In an exam, quote the second half as “factor A through its image”. It is the diagrammatic form of the first isomorphism theorem, and it is what makes Recipes 5–7 rigorous rather than suggestive. (Lean: `rank_le_iff_pinch`.)

8 What the calculus does *not* give you

- **Determinants.** $\det$ is not a composite of wires and boxes: it is an *antisymmetric* multilinear gadget, and antisymmetry is extra structure that plain wires cannot see. You can encode it (with an n -legged antisymmetriser node), but on an exam this costs more than it saves. Compute determinants the usual way.
- **Eigenvalues, characteristic polynomials, Jordan form.** These depend on the *spectrum* of a single square box, which no amount of bending reveals. Diagrams help only for the surrounding bookkeeping (e.g. similarity invariance of the trace, $\text{tr } A = \sum \lambda_i$).
- **Inner products and orthogonality** need a chosen cap ($\cap = \langle -, - \rangle$) and, over $\mathbb{C}$, conjugation — so $A^\top$ becomes A^* and the mirror rule must be stated with conjugate transpose. Say so in your legend if the exam is over $\mathbb{C}$.
- **Non-commutative scalars.** Sliding scalars along wires uses commutativity. Over a non-commutative ring, keep them where they are.
- **Infinite dimensions.** Cups and caps need a finite basis; every bending argument here is finite-dimensional.

9 One-page cheat sheet

Linear algebra	Picture	Move you are allowed
matrix $A \in K^{m \times n}$	box, n wires in, m out	slide along wires
BA	A then B	stretch, no brackets
$A \oplus B$	A above B	slide rows past each other
I	bare wire	insert/delete freely
$A^\top$	bent box	bend, mirror
$(BA)^\top = A^\top B^\top$	mirror the picture	—
$\text{tr } A$	closed loop	rotate the loop
$\text{tr}(AB) = \text{tr}(BA)$	one loop, two boxes	slide round
$\text{rank } A \leq r$	pinch to r wires	narrow the waist
$\text{rank}(AB) \leq \min$	two waists	read the narrower
$\dim \ker + \text{rank} = n$	wires that die + wires that survive	count
$Ax = b$	triangle into box	plug invertible boxes
$x^\top y$	two triangles capped	—
$xy^\top$	two triangles cupped	rank one = one wire
$P^{-1}AP$	A conjugated	cancel P round a loop

Table 1: diagram move $\rightarrow$ lemma to quote

Move	Quote
stretch/slide	associativity of matrix product; $IA = A$
slide rows past each other	block multiplication of $\begin{pmatrix} A & 0 \\ 0 & B \end{pmatrix}$
bend a box	definition of transpose, $(A^\top)_{ij} = A_{ji}$
mirror a picture	$(BA)^\top = A^\top B^\top$
rotate a loop	cyclicity of the trace
narrow a waist	factor through the image; $\text{rank}(BC) \leq \min(\text{rank } B, \text{rank } C)$
delete AA^{-1}	definition of inverse
plug P into both sides	P invertible $\Rightarrow$ same solution set

10 Index of machine-checked rules

All names live in the Lean namespace `ExamToolkit`, file `RequestProject/ExamToolkit.lean`. A diagram with inputs indexed by I and outputs indexed by O is the matrix type `Diagram R I O` = `Matrix O I R`; `seq D E` is “ D then E ”.

Rule / recipe	Statement	Lean name
Sum over paths	$(BA)_{ki} = \sum_j B_{kj} A_{ji}$	seq_path_sum
No brackets	$(CB)A = C(BA)$	seq_assoc
Bare wires	$IA = A = AI$	seq_wires_left/right
Feed a vector	$(BA)x = B(Ax)$	onVec_seq
Interchange	$(B \oplus B')(A \oplus A') = BA \oplus B'A'$	stack_seq
Stacked wires	$I \oplus I = I$	stack_wires
Yanking	$\sum_k \delta_{ik} \delta_{kj} = \delta_{ij}$	yank
Bending transposes	cup-box-cap = $A^\top$	bend_eq_transpose
Mirror	$(BA)^\top = A^\top B^\top$	transpose_seq
Double mirror	$A^{\top\top} = A$	transpose_transpose_diagram
Loop = trace	$\sum_i A_{ii}$	loop_eq_trace
Slide round loop	$\text{tr}(BA) = \text{tr}(AB)$	loop_seq_comm
Conjugation	$\text{tr}(PAP^{-1}) = \text{tr } A$	loop_conj
Scalars multiply	$b \cdot a$ in series	scalarD_seq
Copy then add	= the scalar 2	copy_then_add
Bialgebra	add-copy = copy-cross-add	bialgebraLaw
Hopf	$x + (-x) = 0$	hopfLaw
Pinching	$\text{rank } A \leq r \iff A = BC \text{ through } K^r$	rank_le_iff_pinch
Waist bound	$\text{rank}(BA) \leq \min(\text{rank } A, \text{rank } B)$	rank_seq_le
Mirror preserves rank	$\text{rank } A^\top = \text{rank } A$	rank_transpose_diagram
Wire counting	$\text{rank } A + \dim \ker A = n$	rank_add_nullity
Row operations	$PAx = Pb \iff Ax = b$ (P invertible)	solutions_invariant_of_isUnit
Shifted kernel	$Ax = b \iff A(x - x_0) = 0$	solution_set_eq_coset

The underlying generator calculus (copy, add, scalar, discard, zero, swap, antipode; bialgebra and Hopf laws; linear relations and their converses) is formalised over an arbitrary commutative ring in `RequestProject/GraphicalLinearAlgebra.lean`.

Source. The diagrammatic style is that of the *Graphical Linear Algebra* blog, <https://graphicallinearalgebra.net>.